



LeetCode 1523: Count Odd Numbers in an Interval Range

1. Problem Title & Link

Title: LeetCode 1523: Count Odd Numbers in an Interval Range

Link: <https://leetcode.com/problems/count-odd-numbers-in-an-interval-range/>

2. Problem Statement (Short Summary)

Given two non-negative integers:

low

high

Return the count of **odd numbers** between:

low and high

inclusive.

Example

Input:

low = 3

high = 7

Numbers:

3 4 5 6 7

Odd numbers:

3 5 7

Count:

3

3. Examples (Input → Output)

Example 1

Input:

low = 3

high = 7

Output:

3

Example 2

Input:



low = 8

high = 10

Output:

1

Odd number:

9

Example 3

Input:

low = 1

high = 1

Output:

1

Example 4

Input:

low = 2

high = 2

Output:

0

4. Constraints

$0 \leq \text{low} \leq \text{high} \leq 10^9$

Important:

Range can be very large.

5. Core Concept (Pattern / Topic)

Math

Secondary Topics:

- Counting
- Pattern Observation

6. Thought Process (Step-by-Step Explanation)

Brute Force Idea



Loop through every number.

Example:

low = 3

high = 7

Check:

3 -> odd

4 -> even

5 -> odd

6 -> even

7 -> odd

Count:

3

Works.

Problem

If:

low = 1

high = 1,000,000,000

Looping is too expensive.

Better Observation

Odd and even numbers alternate.

Example:

1 2 3 4 5 6 7 8

Pattern:

Odd

Even

Odd

Even

...



Key Fact

Every pair:

(Odd, Even)

contains:

1 odd number

Count Total Numbers

Formula:

$\text{total} = \text{high} - \text{low} + 1$

Case 1

If total numbers are even:

Example:

3 4 5 6

Count:

4 numbers

Exactly half are odd.

$4 / 2 = 2$

Case 2

If total numbers are odd

Example:

3 4 5 6 7

Count:

5 numbers

Half:

$5 // 2 = 2$

One number remains.

If the starting number is odd:

3

The extra number is odd.

Answer:

$5 // 2 + 1$

$= 3$



Simplified Formula

Count odd numbers from:

low to high

using:

$(\text{high} + 1) // 2 - \text{low} // 2$

This directly counts odds.

7. Visual / Intuition Diagram (ASCII Diagram)

Example:

3 4 5 6 7

Pairs:

(3,4)

(5,6)

Odd count:

2

Remaining:

7

Odd.

Total:

3

8. Pseudocode (Language Independent)

return

$(\text{high} + 1) // 2$

-

$(\text{low} // 2)$

9. Code Implementation

Python

class Solution:

def countOdds(

self,

low: int,



high: int

) -> int:

return (high + 1) // 2 - low // 2

Java

```
class Solution {  
  
    public int countOdds(  
        int low,  
        int high) {  
  
        return (high + 1) / 2  
            - low / 2;  
    }  
}
```

Alternative Interview-Friendly Solution

```
class Solution:  
    def countOdds(self, low, high):  
  
        total = high - low + 1  
  
        if total % 2 == 0:  
            return total // 2  
  
        return total // 2 + (low % 2)
```

10. Time & Space Complexity

Time Complexity

$O(1)$

No loop.

Space Complexity

$O(1)$

Constant memory.



11. Common Mistakes / Edge Cases

Mistake 1

Using a loop.

Violates efficiency.

Mistake 2

Forgetting:

Range is inclusive.

Need:

$\text{high} - \text{low} + 1$

not:

$\text{high} - \text{low}$

Mistake 3

Wrong handling of odd-length ranges.

Edge Cases

Single Odd Number

Input:

$\text{low} = 1$

$\text{high} = 1$

Output:

1

Single Even Number

Input:

$\text{low} = 2$

$\text{high} = 2$

Output:

0

Large Range

Input:

1



1,000,000,000

Still:

$O(1)$

12. Detailed Dry Run

Input:

low = 3

high = 7

Formula:

$(high + 1) // 2$

$= 8 // 2$

$= 4$

low $// 2$

$= 3 // 2$

$= 1$

Answer:

$4 - 1$

$= 3$

Return:

3

13. Common Use Cases (Real-Life / Interview)

Analytics

Count numbers satisfying a pattern in a range.

Mathematics

Parity-based counting.

Number Theory



Even/Odd interval analysis.

Interview Purpose

Tests:

Pattern Recognition

Math Optimization

14. Common Traps (Important!)

Trap 1

Using iteration.

The problem is designed for a formula.

Trap 2

Missing the inclusive range.

Trap 3

Overcomplicating with multiple cases.

A single formula solves everything.

15. Builds To (Related LeetCode Problems)

LC 1523 - Count Odd Numbers



LC 1342 - Number of Steps to Reduce Number to Zero



LC 258 - Add Digits



LC 326 - Power of Three



LC 263 - Ugly Number

These develop mathematical reasoning.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Loop	$O(n)$	$O(1)$	Poor
Mathematical Formula	$O(1)$	$O(1)$	Best



Approach 1: Brute Force

Check every number.

Simple.

Too slow for large ranges.

Approach 2: Mathematical Formula (Chosen)

Uses parity pattern.

Most efficient.

17. Why This Solution Works (Short Intuition)

Half of the integers are odd.

The formula:

$(\text{high} + 1) // 2 - \text{low} // 2$

computes how many odd numbers exist up to high and subtracts how many odd numbers exist before low.

The difference gives the count within the range.

18. Variations / Follow-Up Questions

Follow-Up 1

Count even numbers in a range.

Follow-Up 2

Count multiples of K in a range.

Follow-Up 3

Count prime numbers in a range.

More difficult.

Follow-Up 4

Count numbers divisible by 3 but not by 5.

Interview Takeaway

The key pattern is:

Math + Pattern Observation

Whenever you hear:

Count in Range

Huge Constraints

Even/Odd Pattern



think:

Formula

Many interview problems that appear to require loops can actually be solved in **$O(1)$** by recognizing mathematical patterns.